LoopSparseGS: Loop-Based Sparse-View Friendly Gaussian Splatting

Zhenyu Bao, Guibiao Liao, Kaichen Zhou, Kanglin Liu, Qing Li, and Guoping Qiu, *Senior Member, IEEE*

***Abstract*—Despite the photorealistic novel view synthesis (NVS) performance achieved by the original 3D Gaussian splatting (3DGS), its rendering quality significantly degrades with sparse input views. This performance drop is mainly caused by the limited number of initial points generated from the sparse input, lacking reliable geometric supervision during the training process, and inadequate regularization of the oversized Gaussian ellipsoids. To handle these issues, we propose the LoopSparseGS, a loop-based 3DGS framework for the sparse novel view synthesis task. In specific, we propose a loop-based Progressive Gaussian Initialization (PGI) strategy that could iteratively densify the initialized point cloud using the rendered pseudo images during the training process. Then, the sparse and reliable depth from the Structure from Motion, and the window-based dense monocular depth are leveraged to provide precise geometric supervision via the proposed Depth-alignment Regularization (DAR). Additionally, we introduce a novel Sparse-friendly Sampling (SFS) strategy to handle oversized Gaussian ellipsoids leading to large pixel errors. Comprehensive experiments on four datasets demonstrate that LoopSparseGS outperforms existing state-of-the-art methods for sparse-input novel view synthesis, across indoor, outdoor, and object-level scenes with various image resolutions. Code is available at: https://github.com/pcl3dv/LoopSparseGS**

***Index Terms*—Gaussian splatting, sparse-view synthesis, loop-based, depth alignment, sparse-friendly sampling.**

I. Introduction

NOVEL view synthesis (NVS) aims to generate photorealistic images of 3D scenes from perspectives that were not originally captured [1], [2], [3], [4], [5], [6], [7], [8], which is an essential task in computer vision and graphics field. Recently, 3D Gaussian Splatting (3DGS) [9] has emerged as a promising technique for NVS, as it can efficiently model the highly detailed appearance and geometry of 3D scenes. Such superior performance is usually obtained when large amounts of input images are available. However, in many real-world applications [10], [11], [12], [13], only a few sparse input images are available such as in sports event broadcasting and robotics, where acquiring dense views is often time-consuming and expensive, even impossible.

These sparse inputs bring in serious performance drops to 3DGS-based novel view synthesis. Recent studies [14], [15], [16], [17] have attempted to address this issue. FSGS [15] employs proximity-guided densification and pseudo-view depth regularization but suffers from unreliable 3D Gaussian positions and scale-inconsistent depth constraints. DNGaussian [16] further explores monocular depth regularization but still is hindered by inherent depth inconsistency. While CoR-GS [17] introduces an auxiliary model to guide densification, its effectiveness is constrained by the lack of robust geometric priors, especially under sparse inputs.

However, although these methods have improved the rendering performance of sparse 3DGS, some vital issues remain to be addressed. Firstly, given the sparse input views, the initial Gaussian points provided by Structure from Motion (SfM) [18] can be sparse and inadequate, as shown in Fig. 1 (top left). Secondly, reconstructing the appearance and geometry of scenes becomes an under-constrained and ill-posed issue with insufficient inputs with only the image reconstruction constraints. Previous methods addressed it via scale-inconsistent constraints, which fail to reconstruct the accurate geometry, thus influencing the final rendering performance. Thirdly, the scales of some Gaussians grow to be very large during the optimization process, which can lead to blurring or inaccuracies in corresponding rendering regions illustrated in Fig. 1 (top middle and right).

To address these issues, this paper presents the **LoopSparseGS**, a novel 3DGS framework for precise and robust sparse-input novel view synthesis. LoopSparseGS is built upon a looping mechanism and incorporates a sparse-friendly Gaussian densification strategy with the following considerations. To handle the issues of sparse initialized Gaussian points, the lack of accurate geometry constrained problem, and the excessively large Gaussians in sparse-input scenarios, we have tailored three strategies: a) Progressive Gaussian Initialization (PGI), b) Depth-alignment Regularization (DAR), c) Sparse-friendly Sampling (SFS), respectively, and the detailed them as follows.

a) **The strategy PGI**. As shown in Fig. 1 (bottom left), we observe that those rendered views close to the training

Received 31 July 2024; revised 1 April 2025; accepted 18 May 2025. Date of publication 6 June 2025; date of current version 23 June 2025. The associate editor coordinating the review of this article and approving it for publication was Prof. Sebastian Knorr. *(Corresponding authors: Guibiao Liao; Qing Li.)*

Zhenyu Bao and Guibiao Liao are with the School of Electronic and Computer Engineering, Peking University, Shenzhen 518055, China, and also with the Pengcheng Laboratory, Shenzhen 518055, China (e-mail: zybao@pku.edu.cn; gbliao@stu.pku.edu.cn).

Kaichen Zhou is with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: kc.zhou2020@hotmail.com).

Kanglin Liu and Qing Li are with the Pengcheng Laboratory, Shenzhen 518055, China (e-mail: max.liu.426@gmail.com; lqing900205@gmail.com).

Guoping Qiu is with the School of Computer Science, University of Nottingham, NG7 2RD Nottingham, U.K. (e-mail: guoping.qiu@nottingham.ac.uk).

Digital Object Identifier 10.1109/TIP.2025.3574929

1941-0042 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

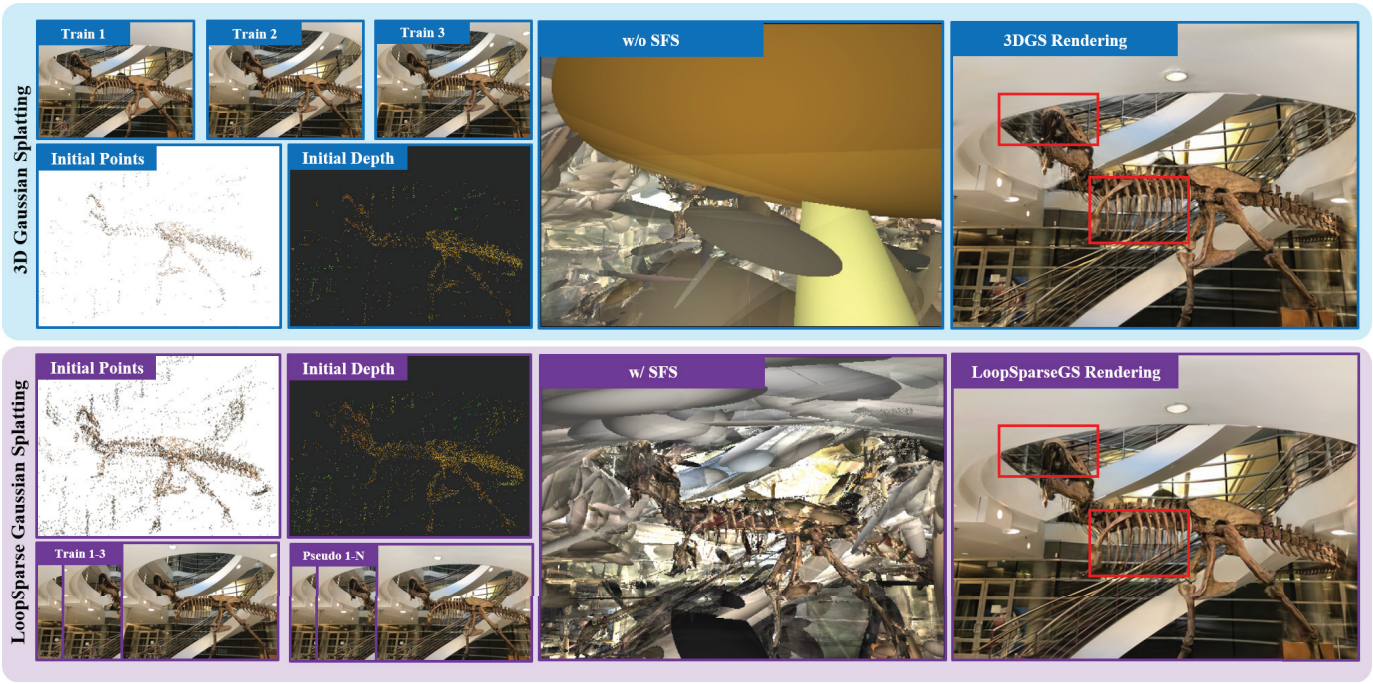


Fig. 1. In scenarios with limited input data, the standard 3D Gaussian Splatting (3DGS) method generates insufficient points and minimal depth constraints for training. Our LoopSparseGS employs additional pseudo-cameras to produce more comprehensive initialization points and richer depth information for 3DGS training. Additionally, we found that the excessively large ellipsoids, damage view rendering quality through blurring. To mitigate the issue, we propose a Sparse Friendly Sampling (SFS) strategy to split oversized ellipsoids. The results are presented in the third column, demonstrating the effectiveness of our method.

views exhibit high visual quality even with sparse input. This observation motivates us to integrate these pseudo-color images, i.e., rendered novel images with training images to generate additional initialized 3D points using SfM. Specifically, we perform multiple rounds of 3DGS training. After each round, we render color images from unseen viewpoints. These rendered images are then combined with the original training images and fed into SfM to iteratively increase initialized Gaussian points, resulting in more comprehensive scene coverage. Since this process is conducted in a loop-based mechanism, we refer to it as Progressive Gaussian Initialization (PGI).

b) **The strategy DAR**. As presented in Fig. 1 (bottom left), the increased number of initialized 3D points from the PGI strategy provides reliable yet sparse scale-consistent depths (hereafter referred to as PGI depth for simplicity). Previous methods [15], [16] rely on dense but inaccurate scale-inconsistent depths to regularize scene geometry, which makes it difficult to impose reasonable depth constraints. To provide a reliable geometric prior and enable effective depth regularization, we integrate scale-consistent PGI depth (with absolute loss) and scale-inconsistent monocular depth (with relative loss) during optimization. Additionally, we introduce a sliding-window mechanism to balance the influence of these two types of depth. Since our method regularizes scene geometry by aligning two different types of depth, we refer to it as Depth-alignment Regularization (DAR).

c) **The strategy SFS**. As illustrated in Fig. 1 (top right), excessively large ellipsoids generated under sparse-input conditions can cause blurring or inaccuracies in the projected regions. To address this issue, we identify and split the Gaussian ellipsoids associated with high-error pixels that carry the largest weights, while penalizing the non-maximum weight ellipsoids to suppress their influence. This strategy yields more detailed geometry and improved rendering quality, as shown in Fig. 1 (bottom right). Since it is specifically designed to handle oversized Gaussians in sparse-input scenarios, we refer to it as Sparse-friendly Sampling (SFS).

The main contributions of this work are as follows:

- We present the LoopSparseGS, a novel 3DGS-based framework for sparse-input novel view synthesis, featuring a looping mechanism to provide denser Gaussian initialization and precise geometry constraints, and a sparse-friendly sampling strategy to address the oversized Gaussian ellipsoids.
- We develop a Progressive Gaussian Initialization (PGI) method to produce dense 3D Gaussian points by incorporating iteratively rendered images with training images into SfM.
- We propose a Depth Alignment Regularization (DAR) approach, which is a novel hybrid optimization framework combining scale-consistent/inconsistent constraints with sliding windows to provide reliable geometric constraints.
- We introduce a Sparse-friendly Sampling (SFS) strategy to address the issue of excessively large Gaussian

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

ellipsoids unique to sparse-input scenes, thus further enhancing the view synthesis quality of the scene.
- Comprehensive experimental results on four datasets demonstrate that our proposed approach outperforms existing state-of-the-art methods in novel view synthesis with sparse-input data, across indoor scenes, outdoor scenes, and object-level scenes.

II. Related Work

A. Novel View Synthesis Using Radiance Fields

Novel view synthesis techniques typically utilize one or more input views to generate images from novel perspectives. Recent advancements in this field have concentrated on employing radiance fields and achieved encouraging progress. For example, Mildenhall et al. [1] introduce Neural Radiance Field (NeRF) that enables novel view synthesis using coordinate-based neural networks. Tremendous following efforts concentrate on improving its rendering quality [19], [20], [21], efficiency [5], [22], [23], [24], and geometry reconstruction [25], [26], [27]. Particularly, Mip-NeRF [19] employs conical frustum instead of single rays to reduce aliasing. Mip-NeRF 360 [20] extends this approach to handle unbounded scenes. Recently, Kerbl et al. [9] achieve a significant breakthrough with the 3D Gaussian Splatting (3DGS) method, which enhances rendering efficiency using explicit Gaussian representations and the differentiable rasterization technique. Building upon the high efficiency of 3D Gaussian Splatting (3DGS) in novel view synthesis, several studies [7], [28], [29], [30], [31], [32] have attempted to improve its applicability in specific scenarios. For example, 3DGS-MCMC [28] maintains constant bandwidth usage by fixing the total number of 3D Gaussians, while Mip-Splitting [29] adopts low-pass filtering and EWA filtering to support zoom-in and zoom-out rendering operations. To enhance the editability of 3DGS, EVSplitting [30] formulates Gaussian splitting as two constrained optimization problems aimed at preserving visual consistency and reducing Gaussian overflow across boundaries.

Although the methods mentioned above demonstrate excellent performance in novel view synthesis, they typically require dense input views for training the radiance fields. When provided with sparse training views, these methods tend to overfit the available training views, resulting in a significant performance drop in novel views.

B. Sparse Novel View Synthesis

In recent years, several NeRF-based studies have been proposed to address sparse-input novel view synthesis. Specifically, RegNeRF [33] introduces geometry and color regularization from unobserved viewpoints, enhancing the quality of sparse-input novel view synthesis. It employs a 2D consistency loss on the depth and color of image patches, ensuring that neighboring regions have similar geometry and appearance. InfoNeRF [34] enhances sparse-input view synthesis by employing regularization techniques based on information theory. Specifically, it applies a sparsity constraint on the density distribution of the ray by minimizing entropy. DS-NeRF [35] utilizes sparse depth cues generated by SfM, to impose depth supervision for sparse NeRF. ViP-NeRF [36] enhances the traditional NeRF framework by incorporating the visibility prior, which enforces multi-view constraints during optimization. This modification involves calculating the visibility of a point and uses these results to regularize the visibility and alpha-blended depth across different views. FreeNeRF [37] incorporates a frequency regularization strategy designed to train the sparse-input NeRF, aiming to regularize the frequency range of NeRF's inputs, and the other to penalize the near-camera density fields. SparseNeRF [38] utilizes a pre-trained depth estimation model to generate pseudo-ground truth depth maps, which are employed for a local depth ranking loss. Besides, SparseNeRF applies a depth smoothness loss to ensure that the rendered depth maps exhibit patch-wise smoothness. Due to the millions of ray samples and implicit MLP queries required by NeRF, NeRF-based sparse reconstruction struggles to achieve real-time rendering, which limits its practical applications.

In addition, some 3DGS-based approaches try to tackle the sparse-input novel view synthesis. For example, Chung et al. [14] adjust the scale of monocular scale-inconsistent depth [39] with SfM depth, utilise dense absolute loss to guide the geometry for sparse-input 3DGS. FSGS [15] introduces the Gaussian unpooling strategy and the pseudo-view depth regularization. The former inserts new Gaussians between neighboring ones guided by a Gaussian proximity score, while the latter regularizes randomly sampled pseudo-view cameras in space using scale-inconsistent monocular depth. However, proximity-based sampling may introduce floating Gaussians in noisy regions, posing challenges to scene geometry reconstruction and novel view synthesis. Likewise, DNGaussian [16] introduces a monocular depth loss and incorporates global-local depth normalization to optimize the parameters of Gaussians. SparseGS [40] proposes generative constraints from a pre-trained diffusion model [41], which guides the 3D Gaussian representation in novel views via Score Distillation Sampling. CoR-GS [17] employs an auxiliary 3D Gaussian Splatting (3DGS) model trained in parallel to constrain the densification process of the primary 3DGS, thereby enabling Gaussian densification at geometrically plausible locations. However, the above methods lack reliable geometric constraints, which limits their ability to represent geometry accurately during optimization. Moreover, they do not provide a robust initialization strategy for sparse scenes or an effective way to handle oversized ellipsoids, which constrains their rendering performance in sparse reconstruction scenarios.

In contrast, our method first introduces a loop-based PGI strategy, which utilises the rendered pseudo-view color images from the last training round to iteratively increase initialized Gaussian points. Furthermore, we propose sliding-window based DAR strategy, which integrates scale-consistent and scale-inconsistent depths for reliable geometry constraints. Finally, we develop a SFS method to handle the oversized ellipsoids tailor for sparse-input scenarios.

Concurrently, Gu et al. [42] tackle sparse-input 3DGS reconstruction by integrating 2D segmentation priors to enforce geometric consistency through multi-view mask

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

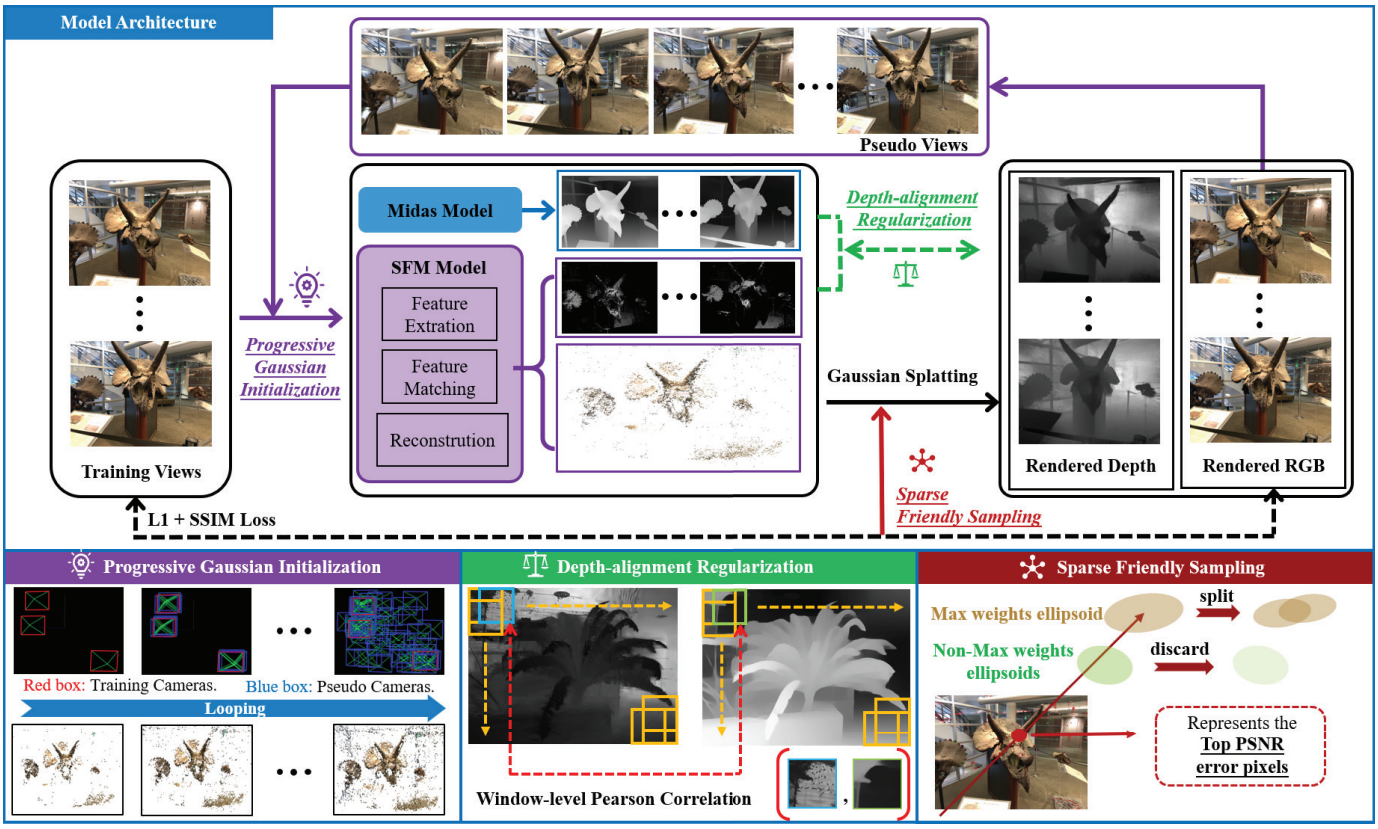


Fig. 2. Overview of the proposed **LoopSparseGS**. The LoopSparseGS features three key components: Progressive Gaussian Initialization, Depth Alignment Regulerizer and Sparse-friendly sampling. Progressive Gaussian Initialization leverages the training view and high-quality pseudo views near the training view to increase the number of the Gaussian initialized points. Depth Alignment Regularizer incorporates the precise SFM depth and monocular depth and provides a sliding window-based manner to align the two scale-invariant depth regularizers. Sparse-friendly sampling slit large Gaussian ellipsoids of large pixels errors to enhance the representation capacity of large pixel areas.

alignment and hierarchical feature regularization. Unlike using multi-granularity semantic constraints, our method incorporates robust initialization, reliable depth constraints, and a customized densification strategy specifically designed for sparse scenarios, leading to a significant enhancement in rendering quality under such conditions. MVPGS [43] introduces a scene-specific pre-trained multi-view stereo (MVS) model to provide dense depth priors, thereby enhancing the geometric consistency of the scene. Instead, our approach does not require a scene-specific pre-trained model and enhances geometric consistency, as validated by various experiments.

III. Methodology

LoopSparseGS facilitates scene novel view synthesis given sparse input images, and the framework is illustrated in Fig. 2. First, initialized sparse point clouds and camera parameters are obtained using Structure from Motion (SfM). We then introduce a loop-based initialization strategy using pseudo-view rendering results to progressively provide denser Gaussian initialization (Section III-B). Second, during the Gaussian optimization, we incorporate depth-alignment regularization to impose reliable and precise geometric constraints (Section III-C). Lastly, we adopt a sparse-friendly Gaussian densification approach to sample effective Gaussians for sparse-input reconstruction quality enhancement (Section III-D). Before introducing our method, we briefly revisit 3D Gaussian Splatting in Section III-A.

A. Preliminaries

3D Gaussian Splatting (3DGS) [9] represents a 3D scene using a set of anisotropic 3D Gaussian primitives, enabling efficient and differentiable rendering via α-blending. The property of i-th Gaussian primitive can be described as $\Theta_i = \{u_i, o_i, s_i, q_i, c_i\}$, where $u_i \in \mathbb{R}^3$ is the center, $o_i \in \mathbb{R}$ is the opacity, $s_i \in \mathbb{R}^3$ is the scaling factor, $q_i \in \mathbb{R}^4$ is the rotation, and $c_i \in \mathbb{R}^3$ is the color. To compute the pixel color C, 3DGS employs differentiable α-blending point-based rendering by blending $\mathcal{N}$ Gaussian points in the front-to-back depth order, which can be written as:

$$C = \sum_{i \in \mathcal{N}} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \tag{1}$$

where $\mathcal{N}$ denotes the set of Gaussian points that overlap with the given pixel, and α_i is calculated by $\alpha_i = o_i f_i^{2D}$, where f_i^{2D} represents the projection function of the i-th Gaussian onto the 2D plane.

3DGS is optimized by projecting 3D Gaussians onto the 2D image plane and employing gradient-based color supervision to minimize the distance between the rendered image

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

$\tilde{I}$ and the ground truth image I. This process can be written as:

$$\mathcal{L}_{color} = (1-\lambda)\mathcal{L}_1(\tilde{I}, I) + \lambda\mathcal{L}_{D-SSIM}(\tilde{I}, I), \tag{2}$$

where λ is set to 0.2 as per [9].

B. Progressive Gaussian Initialization (PGI)

Initialization of Gaussian points is crucial for 3DGS-based novel view synthesis, as it significantly impacts the training convergence quality and speed. The original 3DGS relies on dense input images to generate enough initial Gaussian points. However, in scenarios with limited views, the number of 3D points drops dramatically, potentially compromising reconstruction quality. Considering that rendered views close to the training views exhibit satisfactory visual quality, as illustrated in Fig. 1 (bottom left), we develop a **Progressive Gaussian Initialization (PGI)** approach, which combines the rendered images with the original training images to generate additional initialized points. Instead of performing Gaussian initialization once like vanilla 3DGS, we rely on iteratively refined 3DGS to produce the high-quality pseudo-view color images progressively. The detailed process is shown in Fig. 2.

Given a trained 3DGS and P training views, our PGI module generates a set of pseudo-views for 3DGS initialization by considering both camera positions and orientations. Unlike the previous method [15] that randomly samples pseudo-camera positions within the training camera distribution, we design a progressive camera generation strategy tailored to the loop mechanism (see Fig. 2) to improve the accuracy and robustness of the initialized points. In early loop rounds, pseudo-cameras are placed near the training cameras. As the loop progresses, they gradually expand across the space. Finally, the orientation of each pseudo-camera is determined by averaging the orientations of two adjacent training views, following the approach in [15].

Specifically, before starting a new loop round, we generate 4 new pseudo-views around each training view, resulting in a total of $P \times 4$ new pseudo-views per loop round, where P denotes the number of training views. The camera locations of the pseudo-views are obtained by adding Gaussian noise to the training camera positions. The locations of the generated pseudo views are computed as follows:

$$z_{gl}^{ij} = N(z_{tj} \;\; \varepsilon + \delta \times l)^i, j \in [1, P], i \in [1, 4] \tag{3}$$

where z_g and z_t are locations of the pseudo views and training views, respectively. N is Gaussian noise, j denotes the $j-$th training view, and i denotes the $i-$th pseudo-view generated from one of the training views for each loop round. To ensure the quality of the generated pseudo-view, ε starts from a small value. It gradually enlarges by a rate of δ with loop iterating (l). ε and δ are set to 0.02 and 0.1, respectively. As the number of loop rounds increases, the pseudo view gradually expands the coverage around the training views. To force the view range of the pseudo-images under the coverage of the training views, the locations of pseudo-images should not be out of the bounding box determined by the locations of the training views. For each loop, pseudo-images generated from previous loops are accumulated to train the Gaussians.

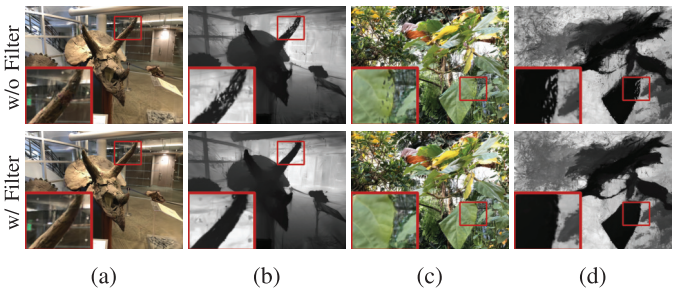


Fig. 3. Illustration of the rendered RGB and depth maps without using filter strategy ("w/o" Filter) and utilizing filter strategy ("w" Filter). (a) Rendered image of *Horns*. (b) Rendered depth of *Horns*. (c) Rendered image of *leaves*. (d) Rendered depth of *leaves*. Without filtering, the rendered depth shows significant holes in the edges of the horn and leaves, while the holes are filled up when using our custom filter strategy.

C. Depth-Alignment Regularization (DAR)

To address overfitting and insufficient constraints, we propose a Depth-Alignment Regularization (DAR) strategy for reliable and accurate geometry regularization. Prior methods typically rely on scale-inconsistent depth to alleviate the under-constrained problem, which makes it difficult to perform effective depth regularization. In contrast, our loop-based PGI method provides an additional advantage: valuable and scale-consistent depth (PGI depth) information derived from the initialized 3D points. To provide a reliable geometric prior and enable effective depth supervision, DAR integrates scale-consistent PGI depth (using absolute loss) and scale-inconsistent monocular depth (using relative loss) during optimization. Given the inherent inaccuracies of SfM-initialized points and the loss imbalance between sparse PGI depth and scale invariant monocular depth, our DAR strategy comprises two components: *Filter Enhancement* and *Sliding Window-based Alignment*. The former improves the accuracy of the SfM-derived PGI depth, while the latter balances the influence of the two types of depth supervision.

Filter Enhancement: We observed that inaccurate depth information directly derived from SfM can result in erroneous rendering, as illustrated in Fig. 3. The *horns* and *leaves* scenes exhibit significant holes at their edges. To enhance depth reliability, we implemented three filtering strategies according to the reliability and visibility of matched points from certain perspectives.

Filter Strategy 1: Firstly, we ignore the depth of 3D points with large match errors according to the SfM key point match report. We use the average pixel match error to find coarse points. The threshold is set to 2. This filter strategy can be described with the following equation.

$$D(p) = \begin{cases} d, & \text{if } \Upsilon(p) < 2, \\ 0, & \text{if } \Upsilon(p) > 2, \end{cases} \tag{4}$$

where Υ is the average pixel match error and d is the depth value. Note that such points are not used for depth map generation but are yet kept for Gaussian initialization. Although such coarse points cannot produce precise depth information they are accurate enough to initialize Gaussian primitives.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

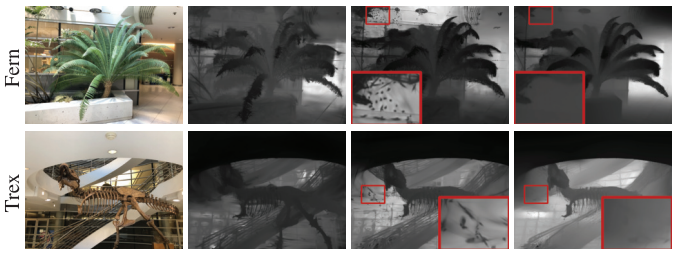


Fig. 4. Illustration of the rendered depth maps using different depth supervision. (a) GT image. (b) Using SfM-derived depth supervision. (c) Using SfM-derived depth and Monocular depth supervision without depth alignment. (d) Using SfM-derived depth and Monocular depth with our depth-alignment strategy.

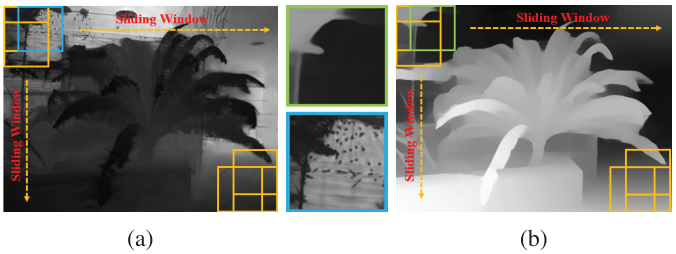


Fig. 5. Illustration of sliding window-based sampling strategy in DAR. (a) Rendered depth map. (b) Monocular depth map provided by Midas. Our method begins by sliding a window to obtain the rendering depth and mono depth with the specified window size. Instead of computing the *Pearson* loss over the whole image, we compute the region of sliding window areas, which enlarge the *Pearson* loss in the misaligned regions between SfM-derived depth and mono-depth, as illustrated in the blue box of (a) and the green box of (b) of the middle area.

Filter Strategy 2: Secondly, considering that pseudo-view images may coincidentally have erroneous regions that satisfy the key points match, we discard points generated solely from pseudo-view. This is illustrated as follows:

$$p = \begin{cases} \text{Keep}, & \text{if } p \subset M(C_i^t, C_j^t) \| M(C_i^t, C_j^g), \\ \text{Discard}, & \text{if } p \subset M(C_i^g, C_j^g), \end{cases} \tag{5}$$

where p is the matched 3D point. C^t and C^g are the training views and pseudo views, respectively. The M indicates that the point is computed from these views. Points are unreliable when they are produced from pseudo-images only rendered by the preliminary trained 3DGS.

Filter Strategy 3: Lastly, considering that the foreground points block the background points in 3D space, we select 3D points to produce the depth images based on visibility provided by the RGB images. For certain view C_i, its corresponding depth D_i of certain 3D point p_j is d, only if p_j derived from view C_i. This can be described under the following conditions:

$$D_i(p_j) = \begin{cases} d, & \text{if } p_j \subset M(C_i, C_k), \\ 0, & \text{other situations}, \end{cases} \tag{6}$$

where $D_i(p_j)$ represents that the depth of point p_j in i-th camera. C_i is the i-th camera. d represents the distance from point p_j to camera C_i.

Sliding Window-Based Alignment: Integrating scale-consistent depth from the projection of SfM 3D points and scale-inconsistent depth could provide a reasonable geometry prior and improve the geometric consistency. However, simultaneously employing both the scale-consistent and scale-inconsistent depths introduces artifacts or noise in regions, as illustrated in (c) of Fig. 4. This phenomenon is attributed to the limitations of the image-level relative Pearson constraints, which struggles to impose sufficient constraints on local regions, thereby hindering the alignment of absolute-scale pixel-level PGI depths.

To balance the two types of depths, we develop a sliding window-based depth regularization, as illustrated in Fig. 5. We slide the window from left to right, and from the top to down with over the monocular depth and render depth. We compute the Pearson distribution loss of the window-covered region rather than across the entire image. This region-based Person distribution loss can effectively work with the L_1 loss generated from sparsely distributed depth values from SfM as they both are effective locally. Therefore, the final loss function (L_a) is formulated as follows:

$$L_a = \lambda_d L_1(D_p, D_c) + \sum_{w=1}^{W} \lambda_p L_p[\chi_w(D_p), \chi_w(D_m)], \tag{7}$$

where D_p, D_c, and D_m denote the rendered depth, SfM-derived depth, and mono-depth, respectively. χ_w represents the w-th window, and W is the total number of windows.

As shown in (d) of Fig. 4, the sliding-window based method effectively balances the scale-consistent PGI depth and scale-inconsistent monocular depth. We attribute this to its adaptive traversal of the depth map, which allows each pixel to be constrained by multiple overlapping windows. This spatial adaptivity mitigates local depth inconsistencies, significantly reducing rendering artifacts and enhancing visual coherence.

D. Sparse-Friendly Sampling (SFS)

The Gaussian ellipsoid densification scheme in 3DGS performs effectively with an large number of training images. However, with extremely sparse input data, some Gaussian ellipsoids may grow excessively large with extremely sparse input data and lead to inferior rendered results, as illustrated in (a) of Fig. 6. This is caused by two primary factors: 1) The initial scale or size of Gaussian ellipsoids is determined by the average distance from its three nearest neighbours. In cases where the sparse initialized point cloud fully covers the entire space, this method can lead to irrationally large scales for some ellipsoids. 2) The extreme sparsity of input data can cause rapid increases in some ellipsoids' scale along certain directions. This leads to overfitting the training views while significantly deteriorating the performance of novel viewing perspective.

One obvious solution is to increase the frequency of Gaussian densification or lower the Gaussian densification threshold. However, these direct strategies would exacerbate the overfitting of Gaussian Splatting in sparse input settings. To address it, we introduce a non-trivial strategy, named **Sparse-friendly Sampling**, which selectively applies densification to Gaussian ellipsoids that adversely affect rendering. In

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Proposed Sparse-Friendly Sampling Methods

$u, o, s, q, c \leftarrow$ The position, opacity, scale, rotation, color of 3D Gaussian primitives
$\hat{C}, C \leftarrow$ Rendered images, Ground Truth
$W \leftarrow$ Gaussian primitive ID maps of the maximum weight
$\mathcal{G}, \mathcal{T} \leftarrow$ The gradient and radius threshold
$\nabla \leftarrow$ 2D Gaussian gradients
$\mathcal{N} \leftarrow$ All of 3D Gaussian primitives.

ToSplitSFS=[]
if IsDensificationIteration($iter$) **then**
  for $k = 1$ to length(TrainingViews) **do**
    $\hat{C}_k, W_k \leftarrow \mathcal{R}(u, o, s, q, c)$
    $\mathcal{L}_p = \mathcal{L}_1(\hat{C}_k, C_k)$
    $\epsilon =$ Quantile$(\mathcal{L}_p, 0.99)$
    $M_k = \mathcal{L}_p > \epsilon$
    SelectSFSID = $W_k \cdot M_k$
    ToSplitSFS.append(SelectSFSID)
    Unique(ToSplitSFS)
  end for
  for j-th Gaussian in $\mathcal{N}$ **do**
    if $((\nabla_j > \mathcal{G})$ || $(j \in$ ToSplitSFS$))$ && $(s_j > \mathcal{T})$ **then**
      SplitGaussian
    else if $(\nabla_j > \mathcal{G})$ && $(s_j > \mathcal{T})$ **then**
      CloneGaussian
    end if
  end for
else
  $\hat{C}_{iter}, W_{iter} \leftarrow \mathcal{R}(u, o, s, q, c)$
  $\mathcal{L}_o \leftarrow 0$
  for l-th Gaussian in $\mathcal{N}$ **do**
    if $l \notin W_{iter}$ **then**
      $\mathcal{L}_o \leftarrow \mathcal{L}_o + \mathcal{L}_1(0, o_l)$
    end if
  end for
  Backward $(\mathcal{L}_o)$
end if

this way, we enhance the representation of Gaussian ellipsoids of large PSNR error area by splitting the oversized Gaussian ellipsoids to produce more Gaussian ellipsoids.

As demonstrated in Fig. 6 excessively large ellipsoids can lead to blurring or inaccuracies in the projected regions. Given a pixel p_1, its color is computed via volume integral in Eq. 1, which results in blurring due to the two reasons: 1) Large ellipsoids with the highest weight often project onto multiple pixels simultaneously, as shown in Fig. 6 blue boxes, and the targeted pixel is disturbed by other pixels corresponding to the same large ellipsoid. 2) Ellipsoids with lower weights may introduce incorrect color contributions by compensating for inaccuracies caused by the excessively large ellipsoid, as shown in Fig. 6 red boxes. This effect is particularly significant



(a) w/o SFS (b) w/ SFS

Fig. 6. Illustration of ellipsoids and their corresponding rendered results.

in sparse scenarios due to under-constrained conditions, further exacerbating pixel blurring.

To relieve it, we adopt a divide-and-conquer strategy guided by rendered pixel errors: Simultaneously splitting high-weight ellipsoids that contribute significantly to pixel-level rendering errors, while regularizing low-weight ellipsoids to suppress their adverse effects. This is different from the previous color-based densification approach [31], which split ellipsoids solely based on accumulated ellipsoid-level weights, thereby overlooking subtle but critical blurring caused by low-weight ellipsoids.

Specifically, we traverse all rendered pixels of the training views and collect pixels with the highest PSNR errors. Since the color of each pixel is derived from multiple ellipsoid primitives through α-blending, we identify the ellipsoid with the largest weight ω (calculated by $\omega = \alpha_i \prod_{j=1}^{i-1}(1-\alpha_j)$) as the primary determinant of the pixel's final color. Subsequently, we apply a splitting procedure to the Gaussian primitives, akin to the method employed in the original 3D Gaussian Splatting (3DGS). This process subdivides large ellipsoids into m smaller ellipsoids to enhance the representation of fine details (in our experiments, $m = 2$). Besides, we introduce opacity regularization during the training process to encourage non-maximum weight ellipsoids along a camera ray to be more transparent. The dual-step operations could effectively handle the blurry and obtain more accurate geometry.

E. Optimization

We summarize our training constraints as follows:

$$\mathcal{L} = \lambda_1 L_1(C_p, C_{gt}) + \lambda_2 L_{\text{D-SSIM}}(C_p, C_{gt}) + L_a + L_o, \tag{8}$$

where C_p and C_{gt} denote the rendered and GT images, respectively. L_1 and $L_{\text{D-SSIM}}$ represent the L1 and SSIM loss. L_a is the depth-alignment loss computed as Eq. 7. L_o is the non-maximum weight regularization:

$$L_o = \frac{\lambda_o}{N} \sum_{n=1}^{N} |\alpha_n|, \tag{9}$$

where N denotes the total number of non-maximum weighted ellipses hit by all pixels in an image. In addition, we compute L_1, $L_{\text{D-SSIM}}$ and L_o for training views and L_a for both training views and pseudo views.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

IV. Experiment

A. Experimental Settings

1) Datasets: To evaluate our sparse-input method, we conduct experiments on four widely used sparse-view datasets for novel view synthesis: LLFF [44], DTU [46], Mip-NeRF360 [20], and Blender [1]. The LLFF dataset [44] consists of real-world scenes captured from roughly forward-facing viewpoints. It contains eight scenes recorded using handheld cellphones, five of which originate from the original LLFF dataset and three from the NeRF dataset. The original datasets provide between 20 and 62 images per scene. Following previous methods [15], [16], we select every eighth image as the held-out testing view, and evenly sample sparse views from the remaining images as the training views. For each scene, three views are utilized to train all the approaches. During evaluation, the image resolutions are set to 1008×756 and 504×378.

The Mip-NeRF360 dataset [20] was obtained from real-world scenes, encompassing nine scenes (five outdoors and four indoors), which were photographed with a 360-degree rotation around the object in the centre. Each scene contains between 100 and 330 images collected using mirrorless digital cameras. As per the protocol in [15], we utilize seven of these scenes for our experiments, employing 24 views with images downsampling rates of 4 and 8 for training all methods.

The DTU dataset [46] is a comprehensive object-level dataset, which comprises 128 scenes captured in a controlled laboratory environment using a structured light scanner. Each scene was scanned from the same set of 49 or 64 camera positions under seven different lighting conditions. In line with [16] and [38], we selected 15 representative scenes exhibiting diverse material appearances and geometries, including challenging non-Lambertian surfaces, and constructed training sets with 3 images per scene. Consistent with the evaluation protocols of prior researches [16], [33], object masks are employed to exclude the background during inference, as evaluating the entire image introduces bias due to background elements.

The Blender dataset [1] is a synthetic dataset rendered with photorealistic images using Blender, consisting of path-traced images from eight object scenes, each characterized by complex geometry and realistic non-Lambertian materials. Six scenes contain views sampled from the upper hemisphere, while the remaining two scenes contain views sampled globally. Following [15] and [33], we use 8 images for training and 25 unseen images for testing.

2) Evaluation Metrics: For quantitative comparisons, we adopt three evaluation metrics: peak signal-to-noise ratio (PSNR), structural similarity index (SSIM) [47], learned perceptual image patch similarity (LPIPS) to assess the visual quality [48].

3) Implementation Details: Our approach is implemented using the PyTorch framework, and utilizes the pre-trained Midas Model [39], [49] for zero-shot monocular depth estimation and the Colmap model for initial camera poses and 3D points. We set the total number of loop iterations as 3 and the number of pseudo cameras generated around one training camera l as 4. In a single loop iteration, we start the densification of Gaussian ellipsoids after 1000 iterations, and the frequency of densification is set to 200 iterations. The 2D grad threshold of densification is set to 0.0005. We set the window length to 32 and the step size of the window sliding to 4 for all datasets. For each loop, 3DGS are trained with 10,000 iterations. The weights of the loss function λ_1, λ_2, λ_o, λ_d, λ_p are set to 0.8, 0.2, 0.05, 0.005 and 0.05, respectively.

4) Efficiency: Our method achieves average rendering speeds (ARS) of 418, 490, 831, and 720 FPS for the LLFF (3 training views), Mip-NeRF360 (24 training views), DTU (3 training views), and Blender (8 training views) datasets, respectively. Here, the average speed is calculated by $ARS = \frac{1}{20}\sum_{i=1}^{20}\frac{M}{T}$, where T is the total inference time of all test images across all scenes in one dataset, the M is the number of all test images, and we tested 20 times to eliminate random error. The training time of the model depends on the number of loops, with a training time of about 10 min per loop and a total training time of about 35-45 min on all datasets. All experiment times were evaluated on a 3090 NVIDIA GPU.

B. Comparison With Existing Methods

1) Comparisons on LLFF: As shown in Tab. I, our method consistently outperforms all baselines across different numbers of input views (3, 6, and 9) and resolutions. Notably, in the extremely sparse 3-view setting, our approach achieves the highest PSNR (20.85/20.19) and competitive SSIM and LPIPS, indicating strong generalization and fidelity in novel view synthesis. With 6 and 9 views, our method further establishes superiority, achieving the best or near-best results across all three metrics. Specifically, it achieves a PSNR of 24.58 and 25.86, and LPIPS of 0.125 and 0.103, respectively, which is second only to the CoRGS, and demonstrates both high reconstruction accuracy and perceptual quality. These results validate the effectiveness of our approach under sparse and dense supervision scenarios. We show the qualitative results in Fig. 7. Existing methods tend to produce artifact and blurry rendered results. In comparison, our approach exhibits fine-grained details such as the leaves (Scenes: *Flower*) and the floor (Scenes: *Room*).

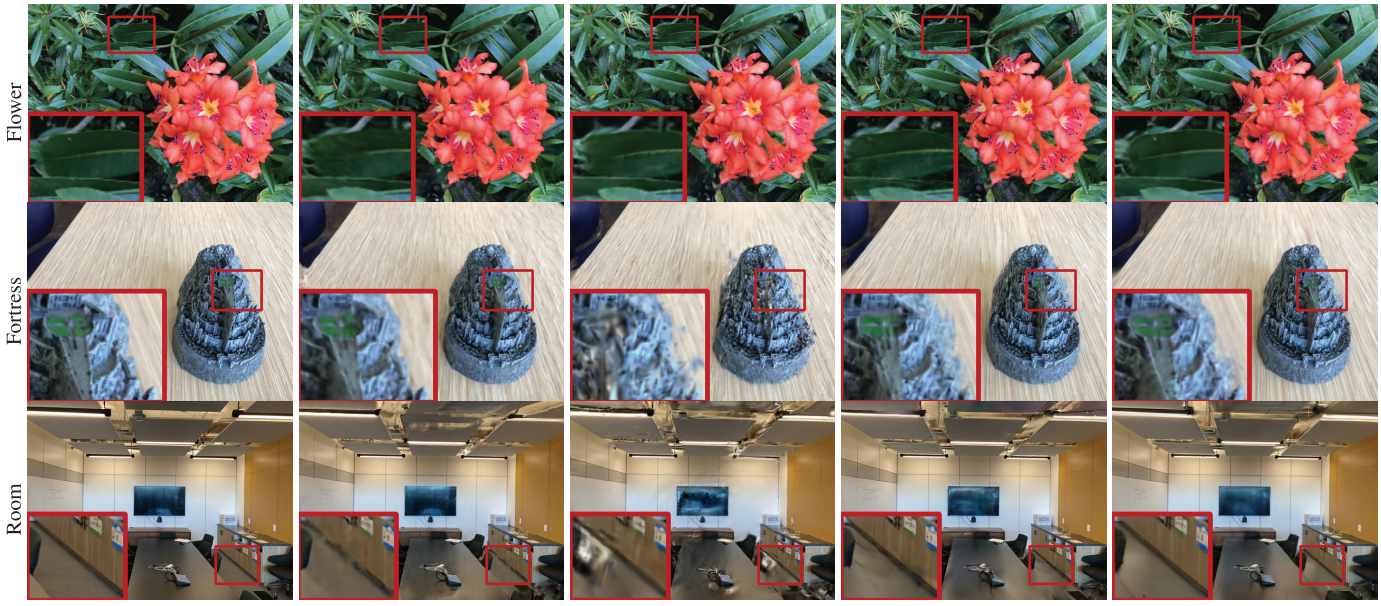
2) Comparisons on Mip-NeRF360: Tab. II presents the quantitative results in complex scenes from Mip-NeRF360. It can be seen that our method also demonstrates significant advantages over existing approaches. Compared to Mip-NeRF requiring dense-input, although methods using regularizations or depth information for sparse-input, such as FreeNeRF and SparseNeRF, enhance rendering quality to some extent, they still encounter a performance bottleneck. Compared to FSGS that incorporates Gaussian unpooling densification technique and monocular depth maps, our proposed method significantly outperforms it with an improvement of 0.39 and 1.02 in PSNR across two resolutions. These demonstrate the effectiveness of our proposed loop-based mechanism and importance-guided sampling strategy. Furthermore, compared to CoR-GS, which employs an auxiliary 3DGS to guide primitive generation for the primary 3DGS, our method offers reliable depth constraints and achieves competitive rendering quality, with PSNR gains of 0.33 (1/8 resolution) and 0.15 (1/4 resolution). Moreover,

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

TABLE I

QUANTITATIVE COMPARISON ON LLFF DATASET [44]

Method	3 Views (1/8 Resolution)			3 Views (1/4 Resolution)			6 Views			9 Views		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Mip-NeRF [19]	16.11	0.401	0.460	15.22	0.351	0.540	22.91	0.756	0.213	24.88	0.826	0.170
DietNeRF [45]	14.94	0.370	0.496	13.86	0.305	0.578	21.75	0.717	0.248	24.28	0.801	0.183
RegNeRF [33]	19.08	0.587	0.336	18.66	0.535	0.411	23.10	0.760	0.206	24.86	0.820	0.161
FreeNeRF [37]	19.63	0.612	0.308	19.13	0.562	0.384	25.13	0.779	0.195	25.13	0.827	0.160
SparseNeRF [38]	19.86	0.624	0.328	19.07	0.564	0.392	24.97	0.784	0.202	24.97	0.834	0.158
3DGS [9]	17.83	0.582	0.321	16.94	0.488	0.402	22.87	0.732	0.204	24.65	0.813	0.159
DNGaussian [16]	19.12	0.591	0.294	18.47	0.578	0.330	22.18	0.755	0.198	23.17	0.788	0.180
FSGS [15]	20.43	0.682	0.248	19.71	0.642	0.283	24.20	0.811	0.173	25.32	0.856	0.136
CoR-GS [17]	20.45	0.712	0.196	19.71	0.675	0.268	24.49	**0.837**	**0.115**	**26.06**	**0.874**	**0.089**
MVPGS [43]	20.54	0.727	**0.194**	19.91	**0.696**	**0.229**	-	-	-	-	-	-
Ours	**20.85**	**0.717**	0.205	**20.19**	0.680	0.274	**24.58**	0.827	0.125	25.86	0.862	0.103



(a) GT (b) FreeNeRF (c) DNGaussian (d) FSGS (e) Ours

Fig. 7. Qualitative results on LLFF datasets. Our method can produce photorealistic results with finer details.

TABLE II

QUANTITATIVE COMPARISON ON MIP-NERF360 DATASET [20]

Method	24 Views (1/8 Resolution)			24 Views (1/4 Resolution)		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Mip-NeRF [19]	21.23	0.613	0.351	19.78	0.530	0.431
DietNeRF [45]	20.21	0.557	0.387	19.11	0.482	0.452
RegNeRF [33]	22.19	0.643	0.335	20.55	0.546	0.398
FreeNeRF [37]	22.78	0.689	0.323	21.39	0.587	0.377
SparseNeRF [38]	22.85	0.693	0.315	21.43	0.604	0.389
3DGS [9]	20.89	0.633	0.317	19.93	0.588	0.401
DNGaussian [16]	22.00	0.683	0.287	21.93	0.668	0.337
FSGS [15]	23.70	0.745	0.230	22.52	0.673	0.313
CoR-GS [17]	23.76	0.752	**0.223**	23.39	**0.727**	**0.271**
Ours	**24.09**	**0.755**	0.226	**23.54**	0.722	0.288

we provide qualitative comparison in Fig. 8. It can be seen that existing methods tend to produce blurred rendered results with incomplete structure, as highlighted by the red boxes around the "plate," "switch," and "window." In contrast, our proposed loop-based and sparse-friendly sampling strategies yield denser initialized Gaussians with effective geometric

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

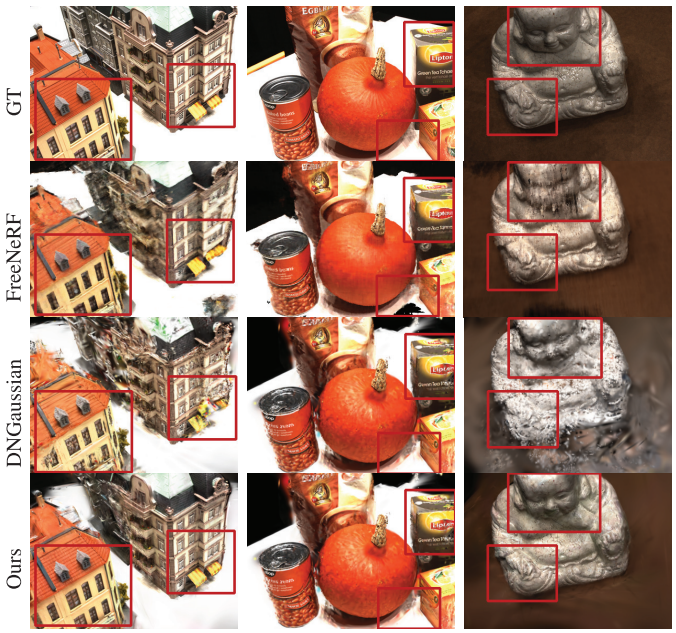


(a) GT (b) 3DGS (c) DNGaussian (d) FSGS (e) Ours

Fig. 8. Qualitative results on Mip-NeRF360 datasets. Our approach can render photorealistic results with more complete structures and finer details.

TABLE III
QUANTITATIVE COMPARISON ON DTU DATASET [46]

Method	3 Views		
	PSNR↑	SSIM↑	LPIPS↓
Mip-NeRF [19]	8.68	0.571	0.353
DietNeRF [45]	11.85	0.633	0.314
RegNeRF [33]	18.89	0.745	0.190
FreeNeRF [37]	19.92	0.787	0.182
SparseNeRF [38]	19.55	0.769	0.201
3DGS [9]	10.99	0.585	0.313
DNGaussian [16]	18.91	0.790	0.176
Cor-GS [17]	19.21	0.853	0.119
RegSegField [42]	18.97	0.833	0.117
MVPGS [43]	20.65	**0.877**	**0.099**
Ours	**20.68**	0.856	0.125



(a) Scan21 (b) Scan31 (c) Scan114

Fig. 9. Qualitative results on DTU datasets.

constraints, resulting in more complete structures and finer details.

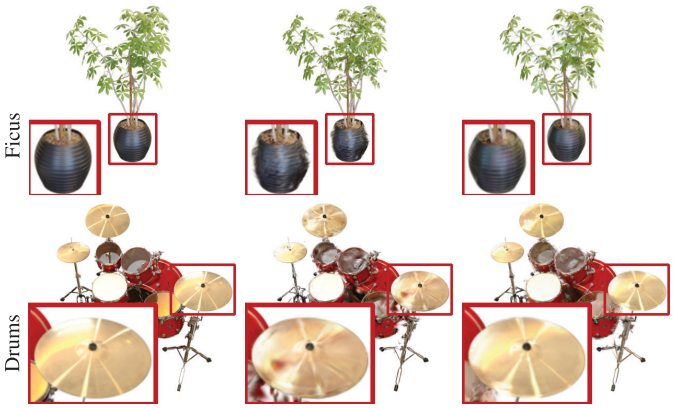
3) Comparisons on DTU: As shown in Tab. III, we present the quantitative results on the DTU 3-view sparse-input setting. For the object-level scenes, our method also achieves the best rendering quality in terms of different metrics, with the significant improvement of 1.77, 1.71, 1.47 and 0.03 with DNGaussian, RegSegField, Cor-GS and MVPGS in PSNR. Notably, MVPGS pre-trained the MVS model on DTU datasets, where our method does not need this scene-specific pre-trained process. Moreover, compared to DNGaussian, which introduces regularization using scale-inconsistent depth, and RegSegField, which incorporates multi-granularity hierarchical semantic information, our method achieves substantial improvements in both PSNR and SSIM. This validates the effectiveness of constraining scene geometry based on reliable depth estimation. In Fig. 9, we present the visual comparisons. Compared to other methods that produce blurry renderings, our approach can capture color details much closer to the ground truth, demonstrating its effectiveness on object-level real-world scenes.

4) Comparisons on Blender: Tab. IV shows the quantitative results on the Blender dataset with an 8-view sparse-input setting. Our method significantly outperforms other approaches on the synthesis scenes, achieving a 0.92 improvement in PSNR compared to the second-best method. Furthermore, Fig. 10 illustrates the rendered results. DNGaussian struggles to achieve precise texture and illumination, whereas our approach accurately captures the geometry of objects and authentic reflections. This demonstrates the effectiveness of our proposed strategies for object-level synthesis scenes.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
QUANTITATIVE COMPARISON ON BLENDER DATASET [1]

Method	8 Views		
	PSNR↑	SSIM↑	LPIPS↓
Mip-NeRF [19]	20.89	0.830	0.168
DietNeRF [45]	22.50	0.823	0.124
RegNeRF [33]	23.86	0.852	0.105
FreeNeRF [37]	24.26	0.883	0.098
SparseNeRF [38]	24.04	0.876	0.113
3DGS [9]	21.56	0.847	0.130
DNGaussian [16]	24.31	0.886	0.088
FSGS [15]	24.64	0.895	0.095
CoR-GS [17]	24.43	0.896	0.084
Ours	**25.56**	**0.906**	**0.075**



(a) GT (b) DNGaussian (c) Ours

Fig. 10. Qualitative results on Blender datasets.

TABLE V
ABLATION STUDY FOR OUR LOOPSPARCEGS

Index	DAR	PGI	SFS	PSNR ↑	SSIM ↑	LPIPS ↓
(a)				19.137	0.637	0.247
(b)	✓			19.466	0.658	0.236
(c)		✓		19.940	0.682	0.217
(d)			✓	19.419	0.638	0.250
(e)	✓	✓		20.565	0.710	0.208
(f)	✓		✓	20.203	0.677	0.225
(g)		✓	✓	20.158	0.692	0.214
(h)	✓	✓	✓	**20.846**	**0.717**	**0.205**

C. Ablation Experiments

In this section, we ablate our method on the LLFF 3-view with 503 × 381 image resolution setting.

1) Effectiveness of Architecture Modules: In Tab. V, we present the ablation results obtained by progressively applying our Depth-alignment Regularization (DAR), Progressive Gaussian Initialization (PGI), and Sparse-friendly Sampling (SFS) strategies. Compared to the baseline (a), each proposed module contributes to improved reconstruction quality, as shown in (b)-(d). Specifically, DAR provides additional depth information to mitigate the under-constrained effect caused by sparse-input, thereby enhancing reconstruction quality. PGI introduces a loop-based initialization strategy and offers denser Gaussians for scene modeling, resulting in a 0.803 increase in PSNR. SFS provides a Gaussian densification strategy suitable for sparse-view training and exhibits a gain of 0.282 PSNR. Furthermore, as shown in (e)-(g), the combination of these strategies yields better results, with all three working together to produce the best performance, as demonstrated in (h).

TABLE VI
ABLATION STUDY FOR THE DEPTH-ALIGNMENT REGULARIZATION

Setting	PSNR↑	SSIM↑	LPIPS↓
wo Depth Loss	20.158	0.692	0.214
SfM Depth Loss	20.205	0.690	0.210
Monocular Depth Loss	20.620	0.706	0.215
SfM + Monocular Depth Loss	20.720	0.709	0.207
Proposed Depth-alignment Loss	**20.846**	**0.717**	**0.205**

TABLE VII
ABLATION STUDY FOR DIFFERENT FILTER STRATEGIES IN PGI

Index	Filter1	Filter2	Filter3	PSNR ↑	SSIM ↑	LPIPS ↓
(a)				20.24	0.6971	0.2216
(b)	✓			20.40	0.7000	0.2209
(c)		✓		20.66	0.7124	0.2049
(d)			✓	20.39	0.7006	0.2168
(e)	✓	✓		20.74	0.7148	0.2053
(f)	✓		✓	20.63	0.7059	0.2164
(g)		✓	✓	20.70	0.7155	**0.2021**
(h)	✓	✓	✓	**20.85**	**0.7173**	0.2046

2) Effectiveness of Depth-Alignment Loss: Tab. VI presents the ablation of our depth-alignment loss through various depth loss configurations. The experiments indicate that both SfM depth loss and monocular depth loss enhance the sparse-input reconstruction quality, with their combination yielding even better results. Moreover, applying our depth-alignment regularization with window-level Pearson correlation loss can further boost the performance. This improvement is attributed to that our window-level operation can effectively align the absolute depth and relative depth constraints and rectify incorrect depth constraints.

3) Effectiveness of Filter Strategies: As shown in Tab. VII, we present the effectiveness of the proposed filter strategies through various configurations. It can be seen that each filter strategy contributes to improved reconstruction quality, as shown in (b)-(d). Moreover, their combination can achieve better results, demonstrating their effectiveness in eliminating unreliable geometric constraints and providing informative supervision for sparse-input view synthesis.

4) Effectiveness of Sparse-Friendly Sampling: In Tab. VIII, we investigate the effectiveness of the non-max weight regularization and max weight densification operations in SFS. The experiments show that both operations enhance the representation of Gaussian ellipsoids, leading to improved PSNR.

5) Number of Looping: To investigate the number of looping in the Progressive Gaussian Initialization (PGI) strategy,

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
ABLATION STUDY FOR SPARSE-FRIENDLY SAMPLING

Setting	PSNR↑	SSIM↑	LPIPS↓
w/o Non-maximum regularization	20.67	0.7156	0.192
w/o Maximum split	20.81	0.7126	0.217
Proposed Sparse-friendly sampling	**20.85**	**0.7173**	**0.205**

TABLE IX
ABLATION STUDY FOR THE NUMBER OF LOOP IN PGI

Loop Number	Point Number	PSNR↑	SSIM↑	LPIPS↓
0	1921	20.20	0.6772	0.2251
1	6370	20.77	0.7112	0.2086
2	8531	20.77	0.7144	0.2065
3	9903	**20.85**	**0.7173**	**0.2046**
4	11078	20.72	0.7154	0.2055
5	11922	20.76	0.7156	0.2054

TABLE X
ABLATION STUDY FOR THE NUMBER OF PSEUDO VIEWS

Number of Pseudo Views	PSNR↑	SSIM↑	LPIPS↓
3	20.812	0.7143	0.2065
6	20.838	0.7150	0.2053
12	**20.846**	**0.7173**	0.2046
24	20.764	0.7166	**0.2036**
48	20.814	0.7163	**0.2036**

TABLE XI
ABLATION STUDY FOR THE THRESHOLD OF EQ. 4

Filter 1 Threshold	PSNR↑	SSIM↑	LPIPS↓
0.5	20.761	0.7126	0.2064
1.0	20.832	0.7158	**0.2041**
1.5	20.830	0.7154	0.2045
2.0	**20.846**	**0.7173**	0.2046
2.5	20.755	0.7157	**0.2041**
3.0	20.769	0.7157	0.2042
5.0	20.819	0.7140	0.2054

we present comparison results in Tab. IX. Increasing the number of loops results in more initialization points, which can enhance rendering performance. However, excessive looping may lead to performance degradation. This is primarily because pseudo-views added at later stages may be distant from the training views, leading to inaccurate initialization points and ultimately impairing performance. Therefore, we selected three loops as the final setting for our experiments.

6) Number of Pseudo Views: To examine the impact of the number of pseudo views in the Progressive Gaussian Initialization (PGI) strategy, we conducted ablation experiments, as shown in Tab. X. The results indicate that adding 12 pseudo images at each Gaussian initialization, equivalent to four times the training views produces better results. Thus, we adopt this as the final configuration.

7) Threshold of DAR Filter Strategy 1 (Eq. 4): To determine the appropriate threshold for Filter strategy 1 in the DAR module, we conducted ablation experiments to evaluate its impact on rendering performance, as shown in the Tab. XI. The results show that a threshold value of 2 yields the best performance. Both higher and lower threshold values lead to degraded rendering quality.

V. CONCLUSION

This paper proposes LoopSparseGS, an innovative 3DGS-based framework for novel view synthesis using sparse input data. In LoopSparseGS, the proposed loop-based strategies, including Progressive Gaussian Initialization (PGI) and Depth-alignment Regularization (DAR), provide denser Gaussians and precise geometric information for effective scene coverage. Additionally, the proposed Sparse-friendly Sampling (SFS) strategy enhances Gaussian densification unique to sparse-input scenes, facilitating the generation of photo-realistic images. Extensive experimental results demonstrate that our approach outperforms existing SOTA methods in sparse-input novel view synthesis across indoor, outdoor, and object-level scenes at various image resolutions.

Our method exhibits limited effectiveness when dealing with sparse inputs containing multiple highlights, primarily due to the difficulty COLMAP encounters in accurately reconstructing highlight-intensive scenes. Employing learning-based pose estimation techniques, such as MASt3R, may alleviate this issue, and this direction will be further explored in future work. In addition, combining our approach with vision foundation models to enhance the quality of unseen color maps could further improve the performance in sparse-input settings. Investigating this combination further is valuable for improving robustness and accuracy in sparse-view tasks, which we plan to explore in future work.

REFERENCES

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, Aug. 2020, pp. 405–421.
[2] K. Zhang, G. Riegler, N. Snavely, and V. Koltun, "NeRF++: Analyzing and improving neural radiance fields," 2020, *arXiv:2010.07492*.
[3] K. Gao, Y. Gao, H. He, D. Lu, L. Xu, and J. Li, "NeRF: Neural radiance field in 3D vision, a comprehensive review," 2022, *arXiv:2210.00379*.
[4] Z. Bao, G. Liao, Z. Zhao, K. Liu, Q. Li, and G. Qiu, "3D reconstruction and new view synthesis of indoor environments based on a dual neural radiance field," 2024, *arXiv:2401.14726*.
[5] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–15, Jul. 2022.
[6] G. Liao et al., "CLIP-GS: CLIP-informed Gaussian splatting for real-time and view-consistent 3D semantic understanding," 2024, *arXiv:2404.14249*.
[7] T. Lu et al., "Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 20654–20664.
[8] K. Zhou et al., "DynPoint: Dynamic neural point for view synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 69532–69545.
[9] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, Aug. 2023.
[10] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 10608–10615.
[11] G. Liao, J. Li, and X. Ye, "VLM2Scene: Self-supervised image-text-LiDAR learning with foundation models for autonomous driving scene understanding," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 4, pp. 3351–3359.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.

[12] K. M. Jatavallabhula et al., "ConceptFusion: Open-set multimodal 3D mapping," in *Proc. Robot., Sci. Syst.*, 2023.
[13] K. Zhou, "Neural surface reconstruction from sparse views using epipolar geometry," 2024, *arXiv:2406.04301*.
[14] J. Chung, J. Oh, and K. M. Lee, "Depth-regularized optimization for 3D Gaussian splatting in few-shot images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 811–820.
[15] Z. Zhu, Z. Fan, Y. Jiang, and Z. Wang, "FSGS: Real-time few-shot view synthesis using Gaussian splatting," in *Proc. Eur. Conf. Comput. Vis.*, Jan. 2023, pp. 145–163.
[16] J. Li et al., "DNGaussian: Optimizing sparse-view 3D Gaussian radiance fields with global-local depth normalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 20775–20785.
[17] J. Zhang et al., "CoR-GS: Sparse-view 3D Gaussian splatting via co-regularization," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2024, pp. 335–352.
[18] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2016, pp. 4104–4113.
[19] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5855–5864.
[20] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-NeRF 360: Unbounded anti-aliased neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 5470–5479.
[21] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Zip-NeRF: Anti-aliased grid-based neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 19697–19705.
[22] A. Chen et al., "TensoRF: Tensorial radiance fields," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2022, pp. 333–350.
[23] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5501–5510.
[24] W. Zhang, R. Xing, Y. Zeng, Y.-S. Liu, K. Shi, and Z. Han, "Fast learning radiance fields by shooting much fewer rays," *IEEE Trans. Image Process.*, vol. 32, pp. 2703–2718, 2023.
[25] P. Wang, L. Liu, Y. Liu, C. Theobalt, T. Komura, and W. Wang, "NeuS: Learning neural implicit surfaces by volume rendering for multi-view reconstruction," 2021, *arXiv:2106.10689*.
[26] L. Yariv, J. Gu, Y. Kasten, and Y. Lipman, "Volume rendering of neural implicit surfaces," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 4805–4815.
[27] D. Azinovic, R. Martin-Brualla, D. B. Goldman, M. Nießner, and J. Thies, "Neural RGB-D surface reconstruction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6280–6291.
[28] S. Kheradmand et al., "3D Gaussian splatting as Markov chain Monte Carlo," in *Proc. Adv. Neural Inf. Process. Syst.*, Apr. 2024, pp. 80965–80986.
[29] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, "Mip-splatting: Alias-free 3D Gaussian splatting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 19447–19456.
[30] Q.-Y. Feng et al., "EVSplitting: An efficient and visually consistent splitting algorithm for 3D Gaussian splatting," in *Proc. SIGGRAPH Asia Conf. Papers*, Dec. 2024, pp. 1–11.
[31] S. R. Bulò, L. Porzi, and P. Kontschieder, "Revising densification in Gaussian splatting," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, Nov. 2024, pp. 347–362.
[32] Z. Yang et al., "Spec-Gaussian: Anisotropic view-dependent appearance for 3D Gaussian splatting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 61192–61216.
[33] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. M. Sajjadi, A. Geiger, and N. Radwan, "RegNeRF: Regularizing neural radiance fields for view synthesis from sparse inputs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 5480–5490.
[34] S. Seo, and B. Han, "InfoNeRF: Ray entropy minimization for few-shot neural volume rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12912–12921.
[35] K. Deng, A. Liu, J.-Y. Zhu, and D. Ramanan, "Depth-supervised NeRF: Fewer views and faster training for free," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 12882–12891.
[36] N. Somraj and R. Soundararajan, "ViP-NeRF: Visibility prior for sparse input neural radiance fields," in *Proc. Special Interest Group Comput. Graph. Interact. Techn. Conf. Conf.*, Jul. 2023, pp. 1–11.
[37] J. Yang, M. Pavone, and Y. Wang, "FreeNeRF: Improving few-shot neural rendering with free frequency regularization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 8254–8263.
[38] G. Wang et al., "SparseNeRF: Distilling depth ranking for few-shot novel view synthesis," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 9065–9076.
[39] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.
[40] H. Xiong, S. Muttukuru, R. Upadhyay, P. Chari, and A. Kadambi, "SparseGS: Real-time 360° sparse view synthesis using Gaussian splatting," 2023, *arXiv:2312.00206*.
[41] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 10684–10695.
[42] K. Gu, T. Maugey, K. Sebastian, and C. Guillemot, "RegSegField: Mask-regularization and hierarchical segmentation for novel view synthesis from sparse inputs," in *Proc. 21st ACM SIGGRAPH Conf. Vis. Media Prod.*, Nov. 2024, pp. 1–10.
[43] W. Xu, H. Gao, S. Shen, R. Peng, J. Jiao, and R. Wang, "MVPGS: Excavating multi-view priors for Gaussian splatting from sparse input views," in *Proc. Eur. Conf. Comput. Vis.*, Nov. 2024, pp. 203–220.
[44] B. Mildenhall et al., "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," *ACM Trans. Graph.*, vol. 38, no. 4, pp. 1–14, Jul. 2019.
[45] A. Jain, M. Tancik, and P. Abbeel, "Putting NeRF on a diet: Semantically consistent few-shot view synthesis," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5885–5894.
[46] R. Jensen, A. Dahl, G. Vogiatzis, E. Tola, and H. Aanæs, "Large scale multi-view stereopsis evaluation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 406–413.
[47] W. Zhou, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, pp. 600–612, 2004.
[48] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 586–595.
[49] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 12179–12188.



Zhenyu Bao received the B.E. degree from the Department of Microelectronic Science and Engineering, Jilin University, in 2019, and the M.S. degree from the School of Electronic and Computer Engineering, Peking University, in 2022. He is currently pursuing the D.S. degree with the School of Computer Science, Peking University. His research interests include novel view synthesis and its applications in AR/XR.



Guibiao Liao received the B.E. degree from Tianjin University, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Electronic and Computer Engineering, Peking University, China. His research interests include 3D computer vision and deep learning.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.



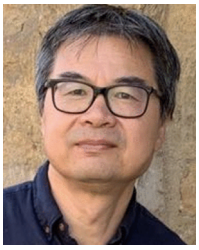
Kaichen Zhou received the master's degree in machine learning from the Department of Computing, Imperial College London, and Ph.D. degree in computer science from the Department of Computer Science, University of Oxford. He is currently a Visiting Lecturer with the School of Computer Science, Peking University. His research interests include computer vision and robotics.



Kanglin Liu received the B.Eng. and Ph.D. degrees from Chongqing University, China, in 2013 and 2018, respectively. From 2016 to 2017, he was a Visiting Scholar with The University of Tennessee and Oak Ridge National Laboratory. From 2018 to 2020, he was a Postdoctoral Fellow with Shenzhen University and the University of Nottingham. He is currently an Assistant Research Fellow with the Peng Cheng Laboratory, China. His current research interests include 3D reconstruction, neural rendering, and head avatar creation.



Qing Li received the Ph.D. degree in computer vision from the University of Nottingham, U.K., in 2020. He is currently an Assistant Researcher with the Peng Cheng Laboratory, China. His research interests include image-based indoor localization, visual SLAM, image processing, 3D reconstruction, and remote image interpretation.



Guoping Qiu (Senior Member, IEEE) is currently a Chair Professor of visual information processing with the University of Nottingham, Nottingham, U.K. He has taught in universities in U.K. and Hong Kong and also consulted for multinational companies in Europe, Hong Kong, and China. He is particularly known for his pioneering research in high dynamic range imaging and machine learning-based image processing technologies. He has published widely and holds several European and U.S. patents. Technologies developed in his laboratory have laid the cornerstone for successful spinout companies that are developing advanced digital photography software enjoyed by tens of millions of global users. His research interests include image processing, pattern recognition, and machine learning.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on June 14,2026 at 07:59:13 UTC from IEEE Xplore. Restrictions apply.